Vinícius dos Santos Tomé - 232006780

Descrição do Trabalho:

O projeto consiste na modelação, simulação e síntese de uma Unidade Lógica e Aritmética (ULA) de 32 bits baseada no conjunto de instruções da arquitetura RISC-V. A ULA foi desenvolvida em VHDL e suporta 14 operações distintas selecionadas por um opcode de 4 bits. Estas operações dividem-se em aritméticas (adição e subtração), lógicas (AND, OR, XOR), deslocamentos de bits (SLL, SRL, SRA) e comparações (SLT, SLTU, SGE, SGEU, SEQ, SNE). O componente possui duas entradas de dados principais (A e B), uma saída de dados (Z) e um sinal de controle (cond) que é ativado (1) quando uma condição de comparação é verdadeira, permanecendo inativo (0) nas restantes operações. A implementação utiliza a biblioteca numeric_std para realizar as conversões seguras e operações aritméticas entre vetores lógicos.

Diferença entre Comparações com e sem Sinal

- **Com sinal (Signed):** O bit mais significativo (MSB) do vetor de 32 bits é interpretado como o bit de sinal, seguindo a convenção de complemento para dois. Isto permite representar números negativos e positivos. Por exemplo, na comparação SLT (Set Less Than), o vetor 1111...1111 (que representa -1) será considerado menor que 0000...0001 (que representa 1).
- **Sem sinal (Unsigned):** Todos os bits são interpretados como magnitude absoluta, representando apenas números inteiros positivos (de 0 a $2^{32} - 1$). Na comparação SLTU, o mesmo vetor 1111...1111 representa o seu valor máximo (4.294.967.295) e será considerado maior que 0000...0001 (1).

Detecção de *Overflow* nas operações ADD e SUB

O *overflow* (transbordo) ocorre quando o resultado de uma operação aritmética excede a capacidade de representação dos 32 bits. Pode ser detectado através da análise lógica dos bits de sinal (MSB, ou seja, o bit 31) dos operandos A, B e do resultado Z:

- **Na Adição (ADD):** Ocorre *overflow* se somarmos dois números positivos e o resultado for negativo (MSB do resultado = 1), ou se somarmos dois números negativos e o resultado for positivo (MSB = 0).
    - Fórmula lógica: Overflow = (A(31) AND B(31) AND NOT Z(31)) OR (NOT A(31) AND NOT B(31) AND Z(31))
- **Na Subtração (SUB):** Ocorre *overflow* se subtrairmos um número negativo de um positivo e o resultado for negativo, ou se subtrairmos um número positivo de um negativo e o resultado for positivo.
    - Fórmula lógica: Overflow = (A(31) AND NOT B(31) AND NOT Z(31)) OR (NOT A(31) AND B(31) AND Z(31))

Código VHDL da ULA (`ulaRV.vhd`)

O código abaixo define a entidade conforme a interface requerida e utiliza o comando sequencial `case` dentro de um `process` para definir o comportamento.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ulaRV is
    generic (WSIZE: natural := 32);
    port (
        opcode : in std_logic_vector(3 downto 0);
        A, B   : in std_logic_vector(WSIZE-1 downto 0);
        Z      : out std_logic_vector(WSIZE-1 downto 0);
        cond   : out std_logic
    );
end ulaRV;

architecture behavioral of ulaRV is
begin
    process (A, B, opcode)
    begin
        -- Valores por defeito para evitar a criação de latches
indesejados
        Z <= (others => '0');
        cond <= '0';

        case opcode is
            when "0000" => -- ADD A, B
                Z <= std_logic_vector(signed(A) + signed(B));

            when "0001" => -- SUB A, B
                Z <= std_logic_vector(signed(A) - signed(B));

            when "0010" => -- AND A, B
                Z <= A and B;

            when "0011" => -- OR A, B
                Z <= A or B;

            when "0100" => -- XOR A, B
                Z <= A xor B;

            when "0101" => -- SLL A, B
                Z <= std_logic_vector(shift_left(unsigned(A),
```

```
to_integer(unsigned(B(4 downto 0)))));

            when "0110" => -- SRL A, B
                Z <= std_logic_vector(shift_right(unsigned(A),
to_integer(unsigned(B(4 downto 0)))));

            when "0111" => -- SRA A, B
                Z <= std_logic_vector(shift_right(signed(A),
to_integer(unsigned(B(4 downto 0)))));

            when "1000" => -- SLT A, B
                if signed(A) < signed(B) then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when "1001" => -- SLTU A, B
                if unsigned(A) < unsigned(B) then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when "1010" => -- SGE A, B
                if signed(A) >= signed(B) then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when "1011" => -- SGEU A, B
                if unsigned(A) >= unsigned(B) then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when "1100" => -- SEQ A, B
                if A = B then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when "1101" => -- SNE A, B
                if A /= B then
                    Z <= (0 => '1', others => '0');
                    cond <= '1';
                end if;

            when others =>
```

```
                null;
        end case;
    end process;
end behavioral;
```

Código do Testbench (`ula_tb.vhd`)

O *testbench* foi estruturado para instanciar a ULA e executar ao menos um teste para cada operação. As operações aritméticas validam cenários com resultados zero, positivos e negativos

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ula_tb is
end ula_tb;

architecture tb_arch of ula_tb is
    -- Declaração do componente ULA
    component ulaRV
        generic (WSIZE: natural := 32);
        port (
            opcode : in std_logic_vector(3 downto 0);
            A, B   : in std_logic_vector(WSIZE-1 downto 0);
            Z      : out std_logic_vector(WSIZE-1 downto 0);
            cond   : out std_logic
        );
    end component;

    -- Declaração de sinais
    signal A_tb, B_tb : std_logic_vector(31 downto 0) := (others
=> '0');
    signal opcode_tb  : std_logic_vector(3 downto 0) := "0000";
    signal Z_tb       : std_logic_vector(31 downto 0);
    signal cond_tb    : std_logic;

begin
```

```
    -- Instanciação da ULA
    uut: ulaRV
        port map (
            opcode => opcode_tb,
            A      => A_tb,
            B      => B_tb,
            Z      => Z_tb,
            cond   => cond_tb
        );

    -- Processo que gera estímulos
    stim_proc: process
    begin
        -- 1. ADD (0000) - Teste Positivo (A=10, B=5 -> Z=15)
        A_tb <= x"0000000A"; B_tb <= x"00000005"; opcode_tb <=
"0000";
        wait for 10 ns; -- avançar o tempo de simulação

        -- 2. ADD (0000) - Teste Zero (A=5, B=-5 -> Z=0)
        A_tb <= x"00000005"; B_tb <= x"FFFFFFFB"; opcode_tb <=
"0000";
        wait for 10 ns;

        -- 3. SUB (0001) - Teste Negativo (A=5, B=10 -> Z=-5)
        A_tb <= x"00000005"; B_tb <= x"0000000A"; opcode_tb <=
"0001";
        wait for 10 ns;

        -- 4. AND (0010) (A=0xFFFF0000, B=0x00FF00FF ->
Z=0x00FF0000)
        A_tb <= x"FFFF0000"; B_tb <= x"00FF00FF"; opcode_tb <=
"0010";
        wait for 10 ns;

        -- 5. OR (0011)[cite: 1]
        A_tb <= x"F0F0F0F0"; B_tb <= x"0F0F0F0F"; opcode_tb <=
"0011";
        wait for 10 ns;

        -- 6. XOR (0100)[cite: 1]
        A_tb <= x"AAAAAAAA"; B_tb <= x"55555555"; opcode_tb <=
"0100";
        wait for 10 ns;

        -- 7. SLL (0101) - Shift Left Logica
        A_tb <= x"00000001"; B_tb <= x"00000004"; opcode_tb <=
"0101";
```

```
        wait for 10 ns;

        -- 8. SRL (0110) - Shift Right Logical
        A_tb <= x"F0000000"; B_tb <= x"00000004"; opcode_tb <=
"0110";
        wait for 10 ns;

        -- 9. SRA (0111) - Shift Right Arithmetic (preserva sinal)
        A_tb <= x"F0000000"; B_tb <= x"00000004"; opcode_tb <=
"0111";
        wait for 10 ns;

        -- 10. SLT (1000) - Set Less Than (Com sinal: -1 < 1 ->
Verdadeiro)
        A_tb <= x"FFFFFFFF"; B_tb <= x"00000001"; opcode_tb <=
"1000";
        wait for 10 ns;

        -- 11. SLTU (1001) - Set Less Than Unsigned (Sem sinal:
0xFFFFFFFF < 1 -> Falso)
        A_tb <= x"FFFFFFFF"; B_tb <= x"00000001"; opcode_tb <=
"1001";
        wait for 10 ns;

        -- 12. SGE (1010) - Set Greater/Equal (Com sinal: -1 >= 1
-> Falso)
        A_tb <= x"FFFFFFFF"; B_tb <= x"00000001"; opcode_tb <=
"1010";
        wait for 10 ns;

        -- 13. SGEU (1011) - Set Greater/Equal Unsigned (Sem
sinal: 0xFFFFFFFF >= 1 -> Verdadeiro)
        A_tb <= x"FFFFFFFF"; B_tb <= x"00000001"; opcode_tb <=
"1011";
        wait for 10 ns;

        -- 14. SEQ (1100) - Set Equal (10 == 10 -> Verdadeiro)
        A_tb <= x"0000000A"; B_tb <= x"0000000A"; opcode_tb <=
"1100";
        wait for 10 ns;

        -- 15. SNE (1101) - Set Not Equal (10 != 5 -> Verdadeiro)
        A_tb <= x"0000000A"; B_tb <= x"00000005"; opcode_tb <=
"1101";
        wait for 10 ns;

        wait; -- Finaliza a simulação
```

```
    end process;
end tb_arch;
```

Tela de simulação:

https://drive.google.com/file/d/1wBUryHPW14Tj35QQzBabyucXCbixAF81/view?usp=drive_link